Kernel: SageMath 10.7

```
def signs(perm):
    s=1
    l=len(perm)
    for i in range(l):
        for j in range(i+1,l):
            if(perm[i]>perm[j]):
                s*=-1
    return s
perm=[1,2,3,5,4]
signs(perm)
```

Out[3]: -1

```
from itertools import combinations
def wedge_product(u,v,w):
    prod={}
    for i,j,k in combinations(range(7),3):
        value=(u[i]*(v[j]*w[k]-v[k]*w[j])-u[j]*(v[i]*w[k]-v[k]*w[i])+u[k]*
(v[i]*w[j]-v[j]*w[i]))
        if value!=0:
            prod[(i,j,k)]=value
    return prod
def coef(w,i,j,k):
    indices=sorted([i,j,k])
    if len(set([i,j,k]))<3:
        return 0
    else:
        return signs([i,j,k])*w.get(tuple(indices),0)


u=[0]*7
v=[0]*7
w=[0]*7

u[0]=1
v[1]=1
w[2]=1
w1=wedge_product(u,v,w)
coef(w1,0,1,2)
```

Out[4]: 1

```
from random import randint
def plucker(w):
    viol=[]
    for i,j in combinations(range(7),2):
        for a,b,c,d in combinations(range(7),4):

            if len({i,j,a,b,c,d})>4: #At 4 it become zero eventually due to
antisymmetry
```

```
                val=(-1*coef(w,i,j,a)*coef(w,b,c,d)+coef(w,i,j,b)*coef(w,a,c,d)-
coef(w,i,j,c)*coef(w,a,b,d)+coef(w,i,j,d)*coef(w,a,b,c))
                if val!=0:
                    viol.append({"I":(i,j),"J":(a,b,c,d),"Value":val})
    return viol
def is_decomposable(w):
    return len(plucker(w))==0
```

```
#Testing

is_decomposable(w1)

counter={}
for i in range(100):

    u=[randint(-5,5) for x in range(7)]
    v=[randint(-5,5) for x in range(7)]
    w=[randint(-5,5) for x in range(7)]
    w2=wedge_product(u,v,w)
    if not is_decomposable(w2):
        counter.append({"u":u,"v":v,"w":w})

if len(counter)==0:
    print("Test passed")
else:
    print("COunter Example :",counter[0])   #Test passed


w3 = {
    (0,1,2): 1,
    (0,3,4): 1,
    (0,5,6): 1
}

is_decomposable(w3)   # NO
plucker(w3)

decom=[]
nondecom=[]
viol_len={}
basis=list(combinations(range(7),3))
for i in range(100):
    w={b:randint(-5,5) for b in basis}
    if is_decomposable(w):
        decom.append(w)
    else:
        nondecom.append(w)
    n=len(plucker(w))
    viol_len[n] = viol_len.get(n,0) + 1



print(len(decom),len(nondecom))  # i get 0 100 means nondecomposable element are
```

```
too much
print(viol_len)
```

Out[6]:
```
Test passed
0 100
{525: 1, 516: 5, 501: 1, 519: 6, 520: 3, 507: 8, 505: 5, 513: 7, 512: 4, 500: 4,
491: 1, 499: 4, 502: 2, 506: 6, 494: 2, 497: 1, 511: 6, 478: 1, 490: 2, 510: 2,
504: 4, 521: 5, 517: 2, 508: 6, 515: 2, 503: 1, 487: 1, 514: 1, 483: 1, 518: 1,
493: 1, 524: 1, 488: 1, 509: 2}
```

```
from itertools import combinations
import numpy as np

basis2 = list(combinations(range(7), 2))

def contraction(ei, w):
    # ei is an integer 0..6
    # w is your dict {(i,j,k): coeff}
    result = {}
    for (a,b,c), coeff in w.items():
        if ei == a:
            result[(b,c)] = result.get((b,c), 0) + coeff
        elif ei == b:
            result[(a,c)] = result.get((a,c), 0) - coeff
        elif ei == c:
            result[(a,b)] = result.get((a,b), 0) + coeff
    return result

def contraction_rank(w):
    matrix = []
    for i in range(7):
        row_dict = contraction(i, w)
        row = [row_dict.get(pair, 0) for pair in basis2]
        matrix.append(row)
    M = np.array(matrix, dtype=float)
    return np.linalg.matrix_rank(M)
ranks = {}
basis = list(combinations(range(7), 3))
for i in range(200):
    w = {b: randint(-5,5) for b in basis}
    w = {k:v for k,v in w.items() if v != 0}
    if w:
        r = contraction_rank(w)
        dec = is_decomposable(w)
        ranks[(r, dec)] = ranks.get((r, dec), 0) + 1

print(ranks)

w1 = {(0,1,2): 1}
print("w1:", contraction_rank(w1), is_decomposable(w1))


w2 = {(0,1,2): 1, (3,4,5): 1}
print("w2:", contraction_rank(w2), is_decomposable(w2))
```

```
w3 = {(0,1,2): 1, (0,3,4): 1}
print("w3:", contraction_rank(w3), is_decomposable(w3))


w4 = {(0,1,2): 1, (1,3,4): 1}
print("w4:", contraction_rank(w4), is_decomposable(w4))


w5 = {(0,1,2): 1, (3,4,5): 1, (0,3,6): 1}
print("w5:", contraction_rank(w5), is_decomposable(w5))
```

Out[7]:
```
{(np.int64(7), False): 200}
w1: 3 True
w2: 6 False
w3: 5 False
w4: 5 False
w5: 7 False
```

```
# Can rank 4 occur? Try systematically
for i in range(50000):
    w = {b: randint(-2,2) for b in basis}
    w = {k:v for k,v in w.items() if v != 0}
    if w:
        r = contraction_rank(w)
        if r == 4:
            print("Found rank 4!", w)
            break
else:
    print("Rank 4 never occurs")
```

Out[11]:
```
Rank 4 never occurs
```